

Current CPU Architecture, Data Representation & RISC/CISC — Detailed Lecture Notes

JKSSB Computer Awareness

October 7, 2026

59.1 von Neumann architecture and the bottleneck

A **von Neumann** (stored-program) computer keeps both instructions and data in the same memory. The processor fetches an instruction, decodes it, and executes it; when the instruction needs a data operand, that operand is read from the same memory. Because a single shared path carries instructions and data, the processor frequently has to wait for memory. This limitation is called the **von Neumann bottleneck**: the rate at which work can proceed is capped by the rate at which instructions and data can be transferred between the CPU and memory over that shared path. Faster memory, a cache, and wider paths all reduce — but do not remove — the bottleneck.

Definition (von Neumann architecture). *A stored-program design in which instructions and data are held together in the same memory and fetched over a shared path between the CPU and memory.*

The important contrast is *same memory for instructions and data*. A system that keeps them physically separate is a different (Harvard-style) design and is not the von Neumann model.

59.2 Addressing modes: immediate addressing

An **addressing mode** describes how an instruction states where its operand is. **Immediate addressing** embeds the operand value directly inside the instruction. For example, an instruction “add 5 to the register” carries the value 5 in the instruction encoding itself. Because the value is already present, no extra memory access is required to obtain the operand.

The near-neighbour contrasts matter for exams:

- *Immediate*: the operand is in the instruction.
- *Register*: the operand is in a named register.
- *Direct*: the instruction names a memory address holding the operand.

The single line to remember is: immediate addressing means the operand is a constant carried with the instruction.

59.3 Cache and the cache-hit ratio

A **cache** is a small, fast memory placed between the CPU and main memory. It holds copies of recently used instructions and data. When the CPU asks for an item and the cache has it, that

access is a **hit**; if the cache does not have it, the access is a **miss** and the item must come from main memory (and is usually placed in the cache for next time).

Definition (Cache-hit ratio).

$$\text{hit ratio} = \frac{\text{number of hits}}{\text{total accesses}} = \frac{\text{hits}}{\text{hits} + \text{misses}}.$$

A **hit** is faster because cache access time is much lower than the access time of main memory. On a hit the CPU avoids the slow main-memory access; on a miss it pays that cost. This is why a higher hit ratio improves average performance.

Example 59.3.1. *If a program makes 900 cache hits and 100 cache misses, the total accesses are $900 + 100 = 1000$ and the hit ratio is $900/1000 = 0.90$ (90%).*

59.4 Data hazards even with register addressing

A **data hazard** is a dependency that prevents an instruction from proceeding correctly in the cycle originally planned. A common case is a read-after-write dependency: an instruction needs the value of a register that an earlier instruction has not yet written. Pipelining is what exposes these dependencies, because several instructions are in progress at once.

Register addressing avoids an extra memory access and removes some hazards caused by memory operands, but it does **not** eliminate all hazards. Two instructions can still depend on the same register. This is the trap: a candidate may reason that “no memory operand means no hazard,” which is false. Hazards arise from dependencies between instructions, whatever kind of operand is used.

59.5 Virtual memory versus cache

Cache and virtual memory are frequently confused, but they solve different problems.

Point	Cache	Virtual memory
Location	Small fast memory next to the CPU.	Uses main memory plus secondary storage as backing store.
Purpose	Reduce average memory access time by keeping recently used blocks close.	Make the memory available to programs appear larger than physical RAM.
Management	Largely hardware.	Operating system with hardware support.

Both involve small “fast” storage for larger “slow” storage, but the cache shortens access time while virtual memory expands the apparent size of memory. They are not the same mechanism and are not interchangeable.

59.6 Instruction cycle: Program Counter and Instruction Register

During the fetch stage the CPU reads an instruction from memory. The **Program Counter (PC)** holds the address of the **next** instruction to be fetched; once the current instruction has

been fetched, the PC is advanced to point at the following one. The **Instruction Register (IR)** holds the instruction that is currently being fetched, decoded, and executed.

The exam trap is the PC. It is easy to say the PC holds “the address of the current instruction,” but at the moment the instruction is executing, the PC already points at the *next* instruction. The correct statement is: the PC holds the address of the next instruction to be fetched.

59.7 Interrupts are processed at instruction boundaries

An **interrupt** is a signal that causes the CPU to suspend the current program temporarily and transfer control to a handler routine, after which execution usually resumes. Interrupts are recognised at **instruction boundaries** — between one instruction and the next — during the program’s execution, not only when the whole program finishes.

The trap is the claim that “an interrupt is processed only after the entire program completes.” That is false. Mid-program interrupts are the normal case: the processor finishes the instruction in progress, saves its state, and services the interrupt before continuing.

59.8 Instruction timing: clock cycles vary

Different machine instructions perform different amounts of work. A simple register-to-register operation may complete in a single clock cycle, while a division, a complex operation, or an instruction that must access slow memory can take many cycles. Instruction timing therefore depends on the instruction, its operands, and the memory system.

The trap is the claim that “all machine instructions take the same number of clock cycles.” The count varies by instruction; averaging over a program gives an average cycles-per-instruction, not a constant per instruction.

59.9 Two’s complement and overflow detection

In an n -bit **two’s-complement** representation the most significant bit is the **sign** bit: 0 for non-negative and 1 for negative values. A negative number is formed by inverting the bits of its magnitude and adding one. With n bits the representable range is -2^{n-1} through $+2^{n-1} - 1$.

Overflow occurs when the arithmetic result cannot be represented in the available bits. The reliable test is the **sign** rule:

Definition (Overflow test). *Overflow occurs when two operands of the **same sign** are added and the result has the **opposite sign**. If the two operands have different signs, no overflow occurs.*

Overflow is **not** the same as a carry out of the most significant bit. A carry can be produced without overflow, and overflow can occur without the final carry telling the full story. Use the sign test, not the carry flag.

Example 59.9.1 (Overflow). *Work in 8-bit two’s complement. Add +7 and +1:*

$$0111 (+7) + 0001 (+1) = 1000.$$

*The operands are both positive, but the sum 1000 has sign bit 1 and represents -8 . Two same-sign operands produced an opposite-sign result, so this is **overflow**.*

Example 59.9.2 (Carry without overflow). *Again in 8 bits, add -1 and -1 :*

$$1111\ 1111 (-1) + 1111\ 1111 (-1) = 1111\ 1110.$$

There is a carry out of the most significant bit, but the discarded 9th bit leaves 1111 1110, which is exactly -2 . The result is correct, so there is *no overflow*. This shows that a carry out of the MSB is not the same as overflow.

59.10 Zero-extension versus sign-extension

Extension widens a value from fewer bits to more bits while preserving its meaning. The rule depends on whether the value is unsigned or signed: zero-extension fills the new leftmost bits with 0 and is used for **unsigned** values; sign-extension copies the sign bit into the new leftmost bits and is used for **signed** values. The common trap is to swap the two.

8-bit pattern	Unsigned	Signed	16-bit sign-extended
0000 0101	5	+5	0000 0000 0000 0101
1111 1011	251	-5	1111 1111 1111 1011

Zero-extending 1111 1011 would give 0000 0000 1111 1011 (value 251), which preserves the *unsigned* value. Sign-extending it gives 1111 1111 1111 1011 (value -5), which preserves the *signed* value. Using the wrong rule changes the value.

59.11 BCD: binary-coded decimal

BCD (Binary-Coded Decimal) represents each **decimal digit** with its own group of **4 bits**. The ten digit codes are:

0	1	2	3	4	5	6	7	8	9
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Example 59.11.1 (BCD encoding). *Encode the decimal number 59 in BCD. Take each digit separately: 5 is 0101 and 9 is 1001. So 59 becomes*

0101 1001 (BCD).

For contrast, the ordinary binary value of 59 is 111011₂. BCD keeps one 4-bit code per decimal digit; it does not pack the whole number as plain binary.

The trap is the claim that BCD is “just the binary of the whole number.” It is not: each decimal digit is coded in 4 bits, and the codes 1010 through 1111 are not valid BCD digits.

59.12 Byte-addressability and effective address vs instruction length

Memory is **byte-addressable** when every individual byte has its own unique address. A word is a group of bytes at consecutive addresses. Because bytes are individually addressable, a word access begins at the address of its first byte.

The **effective address** is the actual location that an instruction uses for its operand. Depending on the addressing mode, it may be given directly or computed at run time from registers and constants. Instruction length is the number of bytes the instruction itself occupies. These are different quantities: a long instruction does not imply a distant operand. The address of the operand is set by the addressing mode, not by the instruction’s size. Confusing the two is the trap; keep “how big is the instruction” separate from “where is the operand.”

59.13 RISC versus CISC

The two broad instruction-set philosophies are **RISC** (Reduced Instruction Set Computer) and **CISC** (Complex Instruction Set Computer).

Aspect	RISC	CISC
Instruction set	Small, simple, regular.	Large and complex.
Instruction length	Fixed.	Variable.
Registers	Many general-purpose registers.	Fewer registers.
Memory access	Load/store architecture: only load and store touch memory.	Instructions may operate directly on memory operands.
Cycles per instruction	Typically one, suited to pipelining.	Often multiple cycles.
Control emphasis	Hardwired control; work shifted to the compiler.	Microcoded control; work done in hardware.
Typical examples	ARM, MIPS, RISC-V.	x86 and older mainframe designs.

RISC keeps instructions simple and fixed-length so they pipeline well, relying on many registers and a load/store discipline. CISC offers powerful, variable-length instructions that can do more in one step, at the cost of more complex decoding. The exam cue is the pair: *RISC = reduced, fixed, many registers, load/store*; *CISC = complex, variable-length, memory operands allowed*.

Exercises

1. State what is shared in von Neumann architecture and define the von Neumann bottleneck.
2. What is immediate addressing, and why does it need no extra memory access for the operand?
3. A program makes 850 cache hits and 150 misses. Compute the cache-hit ratio.
4. Explain why register addressing does not eliminate all data hazards.
5. In 8-bit two's complement, add +7 and +1 and state whether overflow occurs. Justify using the sign test.
6. Encode the decimal number 59 in BCD, and contrast it with the plain binary value.

Solutions

1. Instructions and data share the same memory. The bottleneck is the limited rate at which instructions and data can be transferred between the CPU and memory over the shared path.
2. Immediate addressing carries the operand inside the instruction encoding, so the value is already available and no memory fetch is required for it.
3. Total accesses = $850 + 150 = 1000$, so the hit ratio is $850/1000 = 0.85$ (85%).
4. Register operands avoid a memory access, but two instructions can still depend on the same register (for example a read-after-write dependency), so hazards remain.
5. $0111 (+7) + 0001 (+1) = 1000$, which has sign bit 1 and represents -8 . Two same-sign operands produced an opposite-sign result, so overflow occurs.
6. BCD: $5 \rightarrow 0101$, $9 \rightarrow 1001$, giving 0101 1001. Plain binary of 59 is 111011_2 ; BCD keeps one 4-bit code per decimal digit instead of packing the number as binary.